arXiv:submit/8341803 [cs.CL] 6 Nov 2026

DRD-MOPD: Domain-Routed Delta Optimization for Multi-Teacher On-Policy Distillation

Lin Yao[1,2]
[1]School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
[2]Zhongguancun Academy, Beijing, China
lin.yao@sjtu.edu.cn

Abstract

Multi-teacher on-policy distillation assigns each sample a domain teacher but typically updates shared student parameters. We introduce Domain-Routed Delta Optimization (DRD-MOPD), which specifies which parameter components each domain's samples update. The student is the sum of a frozen base and trainable components initialized from expert-minus-base weights. All components participate in every forward pass; four update modes define the correspondence between sample domains and components, each with independent optimizer states. We analyze these update rules and measure retention of the original expert geometry through angles, projection coefficients, and energy. Prior work such as Open-MOPD distills directly from the base model; 97.02% of its released final checkpoint's increment energy lies outside the span of the original expert directions. In comparison, our OWN checkpoint after 20 updates retains coefficients near one along each component's original direction and pairwise angles of $88.04°$–$92.41°$; its exported BF16 student's increment has 4.90% outside energy, versus 10.52% for the sum of saved FP32 components. The same checkpoint achieves a 32.72% macro score across six benchmarks, compared with 32.44% for its initial task-arithmetic merge. These measurements describe geometry and capability at the evaluated training stages and settings.

1 Introduction

Multi-teacher on-policy distillation (MOPD) aims to consolidate domain experts into one generalist student (10). It builds on knowledge distillation (5, 13) and uses teacher token distributions to supervise responses generated by the student itself (1, 4). A sample's domain determines which teacher provides supervision. In a conventional shared student, however, losses from all domains update the same parameters and adaptive optimizer statistics. The domain assignment therefore specifies the source of supervision while leaving a separate design choice: which trainable parameters should each domain's samples update?

We introduce Domain-Routed Delta Optimization (DRD-MOPD) to make this correspondence explicit. Starting from a common base, task arithmetic represents each expert by its weight difference from that base (6). We retain these differences as full-size trainable parameter components P_j and keep the base B frozen, forming a single dense student $W = B + \sum_{j=1}^{K} P_j$. Their initial sum is the task-arithmetic merge with unit coefficients. All components participate in every forward pass, while training specifies which components each sample can update. Each component maintains its own AdamW history. This lets us track how the components initialized from each expert change within one composed model.

All modes use the same domain-balanced sampling and domain-teacher supervision. Each batch contains equal numbers of prompts from the domains; the current student samples one response per prompt, and the prompt's frozen domain teacher supplies token probabilities for distillation. The four modes specify the correspondence between sample domain i and updated component P_j. OWN uses $j = i$, and OTHER uses all $j \neq i$. ALTERNATE switches between these rules across rollout steps. In SINGLE-COMPONENT, samples

from all domains i jointly update one component selected for that step, with additional sample weighting; the selected component cycles across steps. These rules retain the same composed student and supervising teacher assignments.

This formulation also makes it possible to study what training retains of the original expert geometry. We measure each component's angle to its initial direction, angles between components, projection coefficients on the original expert directions, and energy inside and outside their span. These measurements distinguish retention of original directions from the acquisition of new ones. We also analyze how the update correspondence and independent optimizer histories affect the composed student, including the conditions for equivalence to mixed-gradient SGD or task-wise Adam (18). Since all components enter the same forward model, separate update histories alone do not establish independent domain behavior.

In the three-domain Open-MOPD setting, the original expert components are nearly orthogonal, whereas 97.02% of the released Final checkpoint's increment energy lies outside their span. In one OWN checkpoint after 20 updates, coefficients on each component's original direction remain close to one, and pairwise angles remain between 88.04° and 92.41°, even as individual components rotate from their initial directions and gain energy. The composed student's increment retains coefficients close to $(1, 1, 1)$ and has 4.90% outside energy in the exported BF16 student (10.52% in the saved FP32 component sum) (Section 4.3). On six benchmarks, this checkpoint scores 24.06% on Math, 23.55% on Code, and 50.56% on IF, for a macro score of 32.72%. These are descriptive observations from the evaluated checkpoint and training setting.

Our contributions are threefold. First, we combine expert-delta initialization with four explicit correspondences between domain samples and updated parameter components under shared distillation supervision. Second, retaining these components makes their evolution traceable from each initialization expert; we characterize this evolution through angles, projection coefficients, and energy at component and composed-model levels. Third, we analyze the optimization equivalences and cross-domain interactions induced by the update rules. Task-specific adaptive states are established in AdaTask (18); our contribution concerns their use with expert-initialized components and the specified update correspondences, rather than a new adaptive optimizer.

2 Related Work

Knowledge and on-policy distillation. Knowledge distillation compresses teacher or ensemble predictions into one student (5). Policy Distillation extends this idea to consolidating task-specific reinforcement-learning policies (13). For language models, Generalized Knowledge Distillation (GKD) trains on student-generated sequences to address the mismatch between training and inference prefixes (1). MiniLLM optimizes reverse KL through on-policy updates (4). These works establish the supervision and sampling principles underlying on-policy capability transfer.

Multi-teacher on-policy distillation. MOPD first trains specialized domain teachers through reinforcement learning, then distills their token distributions into one student on its own generated trajectories, assigning each trajectory to its domain teacher (10). Open-MOPD studies uneven recovery and introduces token-share balancing, gap-following allocation, and student reward refresh (3). We retain these mechanisms and study where the resulting parameter updates are accumulated. This separates update routing from the allocation of token-level supervision.

Model merging and task arithmetic. Model soups average models fine-tuned from a shared starting point into one set of weights (15). Task arithmetic defines task vectors as weight differences from a common reference and composes them through addition, scaling, and subtraction (6). TIES-Merging addresses interference by trimming small changes and resolving sign conflicts before merging (17). Work on tangent-space task arithmetic relates successful composition to weight disentanglement and localized functional effects of task directions (11). Our initialization uses task arithmetic with unit coefficients; subsequent training updates the full increment tensors.

Parameter-space MOPD. IM-MOPD initializes from a merge and periodically injects task-vector increments for domains with insufficient recovery (8). Our formulation fixes the base and updates the full expert-increment tensors through domain-routed gradients, maintaining a separate update history for each increment.

Task-specific optimization and modular distillation. PCGrad modifies conflicting task gradients by projection in the shared parameter space (19). AdaTask maintains task-specific gradient statistics, including separate first and second moments for Adam, and sums the resulting updates to shared parameters (18). Under matched clipping and state histories, OWN has the task-wise Adam equivalence described in Appendix B; expert-delta initialization and retained component trajectories distinguish the parameterization studied here from simply separating optimizer moments. Poly-OPD uses a frozen image-generation backbone, shared attention LoRA, and capability-specific FFN adapters under multi-teacher OPD (2). Its selectable adapters differ from our dense student, which always uses the sum of all components. Finally, Δ-MOPD composes teacher-minus-base *logit* shifts into a distillation target (16); our trainable components are *parameter* tensors, and the two approaches operate in different spaces.

3 Method

3.1 Student parameterization

Consider $K \geq 2$ domains indexed by $j \in \{1, \ldots, K\}$. Let B denote the shared base parameters and E_j an initialization expert for domain j, with parameters W_{E_j}. The experts share the parameter layout of B. For each domain, we maintain a full-parameter component P_j, initialized from the corresponding expert-minus-base weights:

$$P_{j,0} = W_{E_j} - B.$$

The base B remains frozen, while each complete tensor P_j is trainable. The student parameters are their sum,

$$W = B + \sum_{j=1}^{K} P_j. \tag{1}$$

At initialization, this gives the task-arithmetic merge of the K experts with unit coefficients (6).

Every input is processed by the full dense network parameterized by W. All components contribute to all forward passes during response generation, training, and inference, including components that receive no update from that sample. The decomposition introduces no separate forward branches or input-adaptive gating. It separates parameter-update histories within one shared network; the inference model requires only the composed weights W.

3.2 Domain-supervised distillation

Each domain i is assigned a frozen supervising teacher T_i. This teacher need not be the initialization expert E_i, and multiple domains may share a supervising checkpoint. Let π_W and π_{T_i} denote the conditional token policies of the student and teacher, respectively.

Each rollout batch contains equal numbers of prompts from all domains. For each prompt x, the student generates one response $y \sim \pi_W(\cdot \mid x)$ by stochastic autoregressive sampling. A rollout step denotes the generation of this batch of responses. For a sample from domain i, teacher T_i provides token probabilities at each sampled prefix $(x, y_{<\ell})$, where ℓ indexes response tokens. The token-level matching target is the reverse KL

$$\mathrm{KL}(\pi_W(\cdot \mid x, y_{<\ell}) \,\|\, \pi_{T_i}(\cdot \mid x, y_{<\ell}))\,.$$

The teacher therefore supplies supervision on the student's own trajectory. The implemented loss uses teacher–student log-probability differences to construct distillation advantages, which are treated as constants when

differentiating a clipped policy surrogate. Domain weights determine the relative loss contributions. The loss approximation and weighting scheme are specified in Appendix C.

3.3 Parameter-update modes

Let i index the sample's domain and j index a trainable parameter component P_j, with $i, j \in \{1, \ldots, K\}$. Each mode specifies the correspondence between domain-i samples and the components P_j that their losses update (Table 1). All modes use the same sampling and domain-teacher supervision defined above.

Let $t = 1, 2, \ldots$ index rollout steps. In SINGLE-COMPONENT, the selected component is $a_t = 1 + ((t - 1) \bmod K)$: samples from all domains i jointly update that same P_{a_t}. The correspondence remains fixed within each rollout step. The base B stays frozen, and every component participates in the student's forward computation.

Table 1: Correspondence between sample domain i and updated parameter component P_j.

Mode	Sample domain	Updated component P_j
OWN	i	$j = i$
OTHER	i	All $j \neq i$
ALTERNATE	i	Odd t: all $j \neq i$; even t: $j = i$
SINGLE-COMPONENT	All i	$j = a_t$, fixed for this step

In OTHER, the full sample gradient is accumulated into each selected component without division by the number of destinations. Consequently, a batch containing all domains supplies updates to all components in both OWN and OTHER, but with different source domains.

SINGLE-COMPONENT additionally weights samples by 1 when $i = a_t$ and by $1/(K - 1)$ otherwise. With equal prompt counts, these coefficients give the selected component's domain and all remaining domains combined equal total weight before domain weighting. The other modes use coefficient 1. These sample coefficients multiply the domain weights in the loss. Figure 1 shows the routing structure for $K = 3$.

3.4 Optimization

Each component maintains independent AdamW first and second moments and an update counter, all initialized to zero (9). Within an optimizer mini-batch, the weighted losses are differentiated at the same current W, and their gradients are accumulated according to the selected mode before any parameters change. Each selected component then receives one AdamW update after independent L2 gradient clipping. Unselected components retain their parameters, moments, and counters and receive no weight decay. The updated components are added to the frozen base according to Eq. (1), yielding the student used for subsequent computation.

Optimizer states persist across rollout steps and changes in the selected components. If a rollout batch is split into multiple optimizer mini-batches, the sampled responses and frozen-teacher targets are reused, while the student-dependent distillation advantages are refreshed at the current parameters before each update. Appendix A gives the gradient-accumulation and state semantics.

At a fixed W and batch, plain SGD without clipping or weight decay makes OWN equivalent to applying the summed domain gradients directly to the student; OTHER multiplies that composite update by $K - 1$. Independent Adam states generally break this equivalence to a shared optimizer. Under matching clipping and state histories with zero weight decay, OWN instead corresponds to task-wise Adam (18). Separate component histories do not imply independent domain behavior: every component update changes the shared W used by all domains. Appendix B derives these relationships.

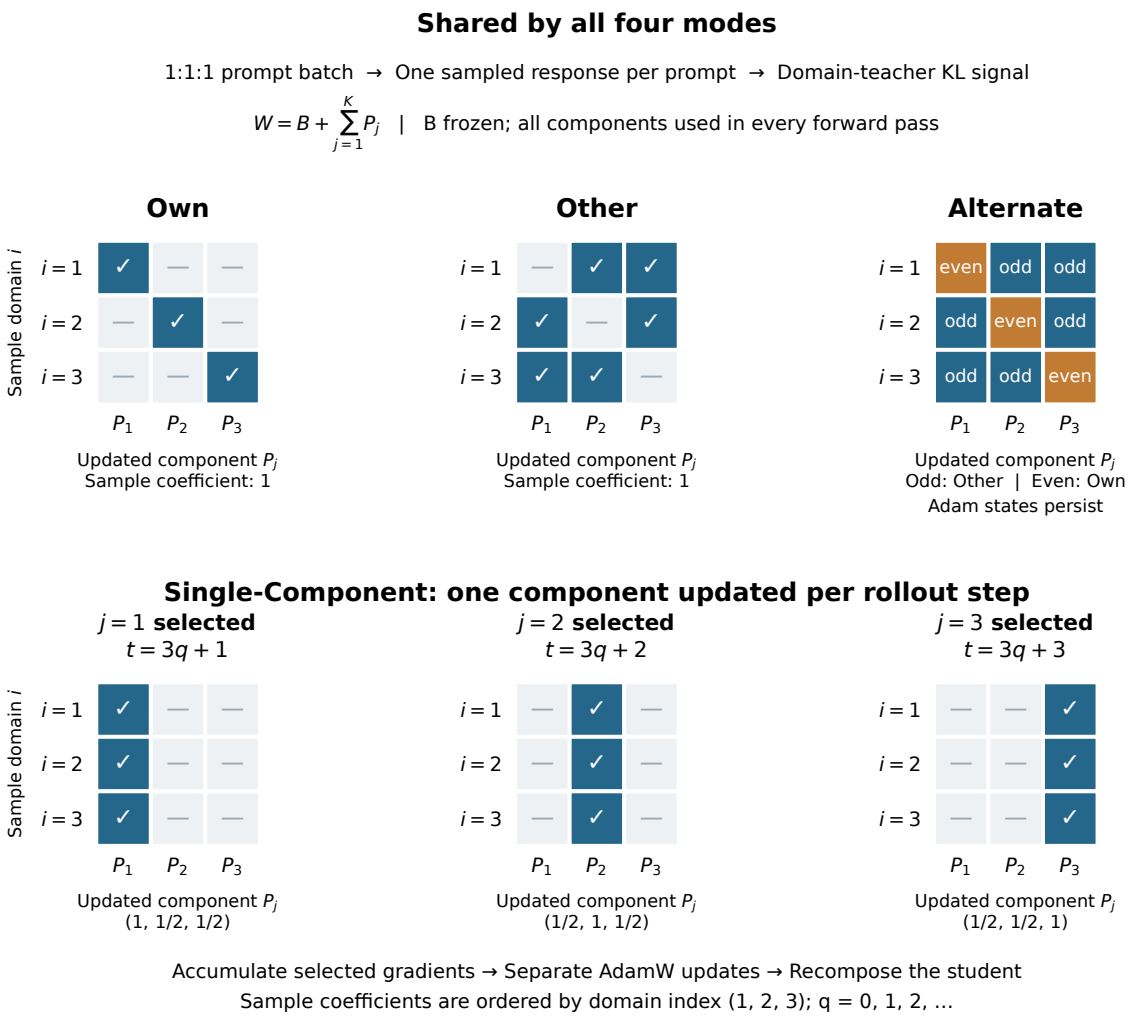


Figure 1: Domain-to-component gradient routing, shown for $K = 3$. Rows identify sample domains and their teachers; columns identify parameter components. Check marks indicate the components receiving each sample's gradient. Both checked and unchecked components contribute to the complete student network; the matrix specifies updates, not forward activation. The bottom row shows the three SINGLE-COMPONENT phases at $t = 3q + 1$, $3q + 2$, and $3q + 3$, where $q = 0, 1, \ldots$. Sample coefficients are ordered by domain index $(1, 2, 3)$.

4 Results

4.1 Experimental setting

We instantiate the general method in the Open-MOPD setting with $K = 3$: domain indices 1, 2, and 3 denote Math, Code, and IF. The base B is SmolLM3-3B MixSFT, and the original Open-MOPD RL-Math, RL-Code, and RL-IF experts initialize P_1, P_2, P_3, respectively. Their unit-coefficient sum gives the TA $[1, 1, 1]$ initial student.

Supervising teachers and selection rationale. Math and Code both learn from a frozen copy of the TA $[1, 1, 1]$ mixture, constructed by adding all three original expert-minus-MixSFT increments to MixSFT. These domains therefore use the merged model as their teacher, replacing their respective original RL experts. IF continues to learn from the original Open-MOPD RL-IF checkpoint. In the notation of Section 3.2,

$$W_{T_{\text{Math}}} = W_{T_{\text{Code}}} = W_{\text{mix}} = B + \sum_{j=1}^{3} (W_{E_j} - B), \qquad W_{T_{\text{IF}}} = W_{E_{\text{IF}}}.$$

The mixture uses unit coefficients in parameter space and is exported as one dense model. Both supervising checkpoints remain frozen throughout distillation, including the mixture initially shared with the student.

We select the mixture for Math and Code because of its stronger scores in the available evaluations and its straightforward construction from public checkpoints, requiring no additional teacher training. Our TA evaluation gives 25.96% on Math and 23.93% on Code, compared with the published domain-RL-teacher reference scores of 24.24% and 21.73% (the RouteRL row in Table 2) (3). These local and published results motivate the teacher choice; they are not a controlled comparison under an identical evaluation run. The source checkpoints and revisions are listed in Appendix C. Thus three domain-teacher assignments use two distinct supervising checkpoints, while three original RL experts supply the component initialization.

Balanced sampling becomes a 1:1:1 prompt quota, and the SINGLE-COMPONENT coefficient for a non-active domain becomes $1/2$.

We compare the original Math, Code, and IF experts, merged models, the released Open-MOPD Final model, and 17 local MOPD training checkpoints. We also analyze one OWN run after 20 updates. Parameter increments are measured relative to MixSFT; their *energy* is the sum of squared parameter changes. Saved FP32 components and the exported BF16 student are measured separately, using FP64 arithmetic. Appendix E gives the model sources and measurement details.

Capability evaluation covers AIME 2024/2025, LiveCodeBench v5/v6, IFEval, and IFBench test. A domain score is the mean of its two benchmarks, and the macro score is the mean across the three domains. The training settings are given in Appendix C.

4.2 Capability evaluation

The TA model starts with a macro score of 32.44%: 25.96% on Math, 23.93% on Code, and 47.43% on IF. The orthogonalized merge scores 24.90%, 24.17%, and 48.01%, respectively, with a macro score of 32.36%. After 20 updates, the OWN student scores 24.06% on Math, 23.55% on Code, and 50.56% on IF, with a macro score of 32.72%. Relative to its TA initialization, the changes are -1.90, -0.37, and $+3.13$ percentage points across the three domains, and $+0.28$ points in the macro score. Thus this checkpoint improves IF while Math and Code decrease. Table 2 places these local evaluations alongside published Open-MOPD reference scores (3). All local rows evaluate a single dense student; RouteOPD and RouteRL select a separate model for each domain.

The evaluation uses a 32,768-token context limit and a 30,000-token output cap. Table 3 reports benchmark scores, sample counts, and length stops. The OWN evaluation passed exact request-ID coverage verification for all 8,101 responses; neither code benchmark contained a test-runner infrastructure error. The domain and aggregate estimators are defined in Appendix D.

Table 2: Capability scores (%), grouped by source. Published results are from Open-MOPD (3); TA, the orthogonalized merge, and OWN at step 20 are our local evaluations under the same benchmark and decoding protocol. The OWN result is from one checkpoint and evaluation run. Worst is the minimum domain score, computed from the displayed values for published rows.

Method / reference	Math	Code	IF	Macro	Worst
Published reference results					
MixSFT (epoch 4)	17.95	17.60	41.46	25.67	17.60
Naive M-OPD	21.26	19.26	43.64	28.05	19.26
Open-MOPD	22.42	21.73	49.58	31.24	21.73
ParamMerge-TA	22.34	22.44	46.60	30.46	22.34
RouteOPD (routed models)	23.15	21.71	49.80	31.55	21.71
RouteRL (routed teachers)	24.24	21.73	51.08	32.35	21.73
Local evaluations of merged models					
TA [1,1,1]	25.96	23.93	47.43	32.44	23.93
Orthogonalized merge	24.90	24.17	48.01	32.36	24.17
Domain-routed parameter methods					
OWN (step 20)	24.06	23.55	50.56	32.72	23.55

Table 3: Six-benchmark evaluation of the TA $[1, 1, 1]$ initialization and OWN after 20 updates: 1,243 prompts and 8,101 responses per model. Math averages correctness over 64 samples per prompt, Code uses ten samples to estimate pass@1, and IF uses strict prompt accuracy with one response. The last two columns count length-stopped responses, which remain in the score denominators.

Benchmark	Prompts	Responses	TA (%)	Own-20 (%)	TA stops	Own stops
AIME24	30	1,920	26.30	22.81	120	205
AIME25	30	1,920	25.62	25.31	138	155
LCB v5	167	1,670	24.25	24.13	245	246
LCB v6	175	1,750	23.60	22.97	280	260
IFEval	541	541	73.20	75.79	20	9
IFBench test	300	300	21.67	25.33	18	3

4.3 Retention of the original expert geometry

We ask whether distillation preserves the original expert directions and their near-orthogonal arrangement. Table 4 reports angles to explicit reference vectors, projection coefficients on the original expert directions, and energy. For a composite checkpoint, the measured vector is its increment $u = W - B$. For a saved component, it is $P_{j,s}$ at step s, where $j = 1, 2, 3$ denotes Math, Code, and IF. The table uses the same columns throughout: each angle compares the row's vector with its named reference, while energy is the row vector's squared L2 norm. The outside fraction measures its energy outside the span of $P_{1,0}, P_{2,0}, P_{3,0}$.

The TA starting point contains three nearly orthogonal directions. The original Math–Code, Math–IF, and Code–IF angles are $88.80°$, $89.99°$, and $89.98°$. Each initial component has unit coefficient on its own direction and zero on the others. Their TA sum has coefficients close to $(1, 1, 1)$ and only 0.0576% outside energy after BF16 export. These quantities establish the reference for assessing what training retains.

Open-MOPD Final differs substantially from the original mixture. Final's increment is $84.40°$ from the TA increment. Its projection coefficients on the original Math, Code, and IF directions are $(0.0258, 0.0319, 0.3489)$, and 97.02% of its energy lies outside their span. These measurements describe a single composite vector. Final's checkpoint does not supply three separately trained components, so it does not establish their pairwise angles.

Own retains the original projections and near-orthogonal arrangement. After 20 updates, the Math, Code, and IF components have rotated $18.89°$, $9.77°$, and $37.43°$ from their respective initial directions. Their coefficients on those directions remain 1.0015, 1.0036, and 0.9999; all cross-direction coefficients have magnitude below 0.0082. The pairwise angles change from $(88.80°, 89.99°, 89.98°)$ to $(88.04°, 92.41°, 91.25°)$ in Math–Code, Math–IF, Code–IF order. Thus, the components adapt while retaining almost their original projection strengths and remaining approximately mutually orthogonal. Their energies grow by 12.07%, 3.71%, and 58.55%, respectively; the outside fractions are 10.48%, 2.88%, and 36.92%.

The composed Own student remains close to its TA initialization. The row "OWN, step 20" measures the exported BF16 student, using u_{TA} as the angle reference. Its $12.86°$ angle therefore compares $W_{\mathrm{Own},20} - B$ with $W_{\mathrm{TA}} - B$. Its three coefficients remain close to one, energy grows from 2.583032 to 2.715631 (5.13%), and 4.90% lies outside the original expert span. This connects component-level retention to the actual composed student. For comparison, summing the saved FP32 components in FP64 gives 10.52% outside energy and coefficients $(0.9986, 0.9985, 1.0002)$; its angle to its initial sum is $18.93°$. The BF16 and FP32 measurements therefore support retention of the original projections but give different residual energies. The 4.90% figure describes the exported model used for capability evaluation, not the full-precision training components. The comparison is descriptive: this Own run is at step 20, whereas the released Final model is at step 200 and uses a different initialization and training protocol.

The table's outside fractions use total increments from MixSFT, rather than only the new changes made during training. A small new update can lie almost entirely outside the original span while leaving most of the initial mixture intact. Appendix E reports these additional-update measurements, the historical TA-initialized trajectory, and precision controls.

Table 4: Angles, coefficients, and energy relative to the original expert geometry. Each angle compares the row vector with the named reference. Composite rows measure $u = W - B$ from BF16 checkpoints; component rows measure saved FP32 tensors. $P_{j,0}$ and $P_{j,20}$ denote initialization and Own step 20, with $j = 1, 2, 3$ for Math, Code, and IF. Coefficients are ordered $(1, 2, 3)$, and energy is the squared L2 norm. The final group repeats only pairwise-angle comparisons. MixSFT has no increment direction. All measurements use FP64 arithmetic.

Vector / checkpoint	Angle reference	Angle (°)	Original-direction coefficients	Energy	Outside (%)
Composite models relative to the original TA mixture					
MixSFT	u_{TA}	—	(0.0000, 0.0000, 0.0000)	0.000000	—
TA $[1, 1, 1]$	u_{TA}	0.00	(0.9990, 0.9993, 1.0000)	2.583032	0.0576
Orthogonal merge	u_{TA}	1.66	(0.9969, 0.9999, 0.9995)	2.581603	0.0576
Open-MOPD Final	u_{TA}	84.40	(0.0258, 0.0319, 0.3489)	1.677080	97.0211
OWN, step 20	u_{TA}	12.86	(0.9992, 0.9995, 1.0002)	2.715631	4.8999
Individual components: initialization and Own step 20					
$P_{1,0}$	$P_{1,0}$	0.00	(1.0000, 0.0000, 0.0000)	0.671449	0.0000
$P_{1,20}$	$P_{1,0}$	18.89	(1.0015, 0.0031, 0.0001)	0.752461	10.4794
$P_{2,0}$	$P_{2,0}$	0.00	(0.0000, 1.0000, 0.0000)	1.476998	0.0000
$P_{2,20}$	$P_{2,0}$	9.77	(0.0007, 1.0036, 0.0002)	1.531823	2.8788
$P_{3,0}$	$P_{3,0}$	0.00	(0.0000, 0.0000, 1.0000)	0.394056	0.0000
$P_{3,20}$	$P_{3,0}$	37.43	(-0.0037, -0.0082, 0.9999)	0.624788	36.9234
Pairwise angles: initialization and Own step 20					
$P_{1,0}$	$P_{2,0}$	88.80	—	—	—
$P_{1,20}$	$P_{2,20}$	88.04	—	—	—
$P_{1,0}$	$P_{3,0}$	89.99	—	—	—
$P_{1,20}$	$P_{3,20}$	92.41	—	—	—
$P_{2,0}$	$P_{3,0}$	89.98	—	—	—
$P_{2,20}$	$P_{3,20}$	91.25	—	—	—

5 Conclusion

We presented DRD-MOPD for multi-teacher on-policy distillation. A shared dense student starts from the sum of the original expert task vectors and a frozen base; four routing rules govern how domain-supervised gradients update these vectors and their independent optimizer states. The optimization analysis establishes relationships to mixed-gradient SGD and task-wise Adam, while distinguishing separate update histories from independent behavior. Weight measurements characterize which parts of the initial expert geometry survive distillation. The original components are nearly orthogonal. In one Own checkpoint after 20 updates, their angles to their respective initial directions are $18.89°$, $9.77°$, and $37.43°$, while their original-direction coefficients remain close to one and their pairwise angles remain near $90°$. The exported BF16 student is $12.86°$ from its TA initialization with 4.90% outside energy. The released Open-MOPD Final increment is $84.40°$ from TA and has 97.02% outside energy. These are descriptive comparisons across different training settings. They show retention of projection strengths and near-orthogonality in the inspected Own components alongside new out-of-span changes. The same checkpoint reaches a 32.72% macro score on six benchmarks, with higher IF and lower Math and Code scores than its TA initialization. Geometry and capability therefore provide complementary descriptions of this checkpoint.

6 Limitations

Published reference scores and the local evaluations use different training and evaluation conditions; historical MOPD trajectories also differ in objectives and budgets. These comparisons are therefore descriptive. The component-level analysis and routed-model benchmark evaluation cover one OWN checkpoint at step 20 with one training seed and one evaluation run. Assessing the stability of its angles and energy changes requires longer trajectories and repeated runs.

The geometric analysis uses the span of three complete expert increments. Establishing a causal relationship between these measurements and capabilities requires controlled interventions. BF16 precision further limits the parameter changes visible in exported checkpoints.

Full-parameter components and independent optimizer moments increase training memory and checkpoint storage. The implementation covers three domains with FSDP1 and two distinct supervising checkpoints, with Math and Code sharing the TA teacher. Scaling to more experts and other sharding schemes remains future work.

References

[1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2306.13649.

[2] Siming Fu, Haojun Xu, Ruizhe He, Zheming Fu, Hualiang Wang, Jie Huang, Xiaoxiao Ma, Mingchen Zhong, Weihu Huang, Xiaoxuan He, Linjiang Huang, and Si Liu. Poly-OPD: Heterogeneous multi-teacher on-policy distillation for capability-selectable flow models. *arXiv preprint arXiv:2608.04349*, 2026. URL https://arxiv.org/abs/2608.04349.

[3] Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-MOPD: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation. *arXiv preprint arXiv:2608.19098*, 2026. URL https://arxiv.org/abs/2608.19098.

[4] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024. URL https://proceedi

ngs.iclr.cc/paper_files/paper/2024/hash/8ac015d409635f196f9e3e9dcfb9a94e-Abstract-Conference.html.

[5] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. URL https://arxiv.org/abs/1503.02531.

[6] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2212.04089.

[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. URL https://arxiv.org/abs/2403.07974.

[8] Seonghyeon Kim, Chaeyun Jang, Noah Lee, Boseop Kim, and Juho Lee. No pain, more gain: Iterative merging for effective multi-teacher on-policy distillation. *arXiv preprint arXiv:2609.34745*, 2026. URL https://arxiv.org/abs/2609.34745.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[10] Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. MOPD: Multi-teacher on-policy distillation for capability integration in LLM post-training. *arXiv preprint arXiv:2606.30406*, 2026. URL https://arxiv.org/abs/2606.30406.

[11] Guillermo Ortiz-Jimenez, Alessandro Favero, and Pascal Frossard. Task arithmetic in the tangent space: Improved editing of pre-trained models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d28077e5ff52034cd35b4aa15320caea-Abstract.html.

[12] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following. *arXiv preprint arXiv:2507.02833*, 2025. URL https://arxiv.org/abs/2507.02833.

[13] Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. Policy distillation. In *International Conference on Learning Representations*, 2016. URL https://arxiv.org/abs/1511.06295.

[14] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

[15] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 23965–23998, 2022. URL https://proceedings.mlr.press/v162/wortsman22a.html.

[16] Yuanda Xu, Hejian Sang, Zhengze Zhou, Shayan Mohajer Hamidi, Xiaomin Li, Rohit Jain, and Alborz Geramifard. Composing what each teacher learned: Delta distillation for multi-teacher on-policy learning. *arXiv preprint arXiv:2610.10460*, 2026. URL https://arxiv.org/abs/2610.10460v2. Version 2.

[17] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A. Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1644c9af28ab7916874f6fd6228a9bcf-Abstract-Conference.html.

[18] Enneng Yang, Junwei Pan, Ximei Wang, Haibin Yu, Li Shen, Xihua Chen, Lei Xiao, Jie Jiang, and Guibing Guo. AdaTask: A task-aware adaptive learning rate approach to multi-task learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 10745–10753, 2023. doi: 10.1609/aaai.v37i9.26275. URL https://arxiv.org/abs/2211.15055.

[19] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://arxiv.org/abs/2001.06782.

[20] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. URL https://arxiv.org/abs/2311.07911.

A Distributed Routing and State Semantics

The implementation materializes the complete student weights W by adding all parameter components to the base and performs ordinary dense forward and backward passes on this model. After capturing the gradient with respect to W, the optimizer accumulates it into the selected components' gradient buffers and rematerializes W after their updates. Routing does not alter the execution path through the network layers.

Domain routing changes micro-batch grouping while retaining Open-MOPD's clipped policy surrogate (3, 14). Let ℓ_i be the sum of inherited token losses for sample i. For an original micro-batch indexed by m, let N_m be its effective response-mask sum, including the denominator's numerical stabilizer. A sample in that micro-batch contributes ℓ_i/N_m.

A domain-homogeneous packed micro-batch may collect samples from several original micro-batches. Let $m(i)$ identify sample i's original micro-batch. For a packed micro-batch indexed by p, let N_p be the corresponding effective response-mask sum, with the same numerical stabilizer. Multiply each contained sample's advantages by the normalization factor

$$\kappa_i = \frac{N_p}{N_{m(i)}}. \tag{2}$$

The inherited surrogate is positively homogeneous in its advantages, including the negative-advantage clipping branch. The rescaling therefore changes the sample numerator to $\kappa_i \ell_i$, and division by N_p recovers $\ell_i/N_{m(i)}$ exactly in real arithmetic. The original gradient accumulation factor and rank averaging are retained. This corrects the regrouping denominator while preserving teacher and question weights. Original response masks and any per-question format masks determine both denominators.

Ranks synchronize the maximum number of packed micro-batches for each domain and execute those domains in a fixed order. Missing local slots carry zero question coefficients, while retaining a valid sequence and attention mask. This maintains collective alignment without creating a gradient contribution. Gradients are captured before switching domains. Branch norms are computed across local FSDP shards before a single clipping operation for each component.

An active component is one selected by a processed domain, even when its resulting gradient is numerically zero. Such a component follows AdamW's normal momentum and decay update. An inactive component is not selected at all and skips the update. Thus a zero gradient and an inactive component have different state transitions. The supported training path uses FSDP1 with `use_orig_params=False`, full sharding on multiple GPUs, and no sharding on one GPU. Dynamic actor batching and sequence parallelism are disabled in this implementation. The base, FP32 components, and optimizer states are stored as local shards in host memory; one FSDP model holds the composite weights for computation. Checkpoint resume restores the component states and the absolute rollout step so that the routing schedule continues from the saved step.

B Analysis of Student Updates

This appendix derives the update properties summarized in Section 3.4, covering gradient accumulation under SGD, independent Adam histories, and effects on the composite student.

For the derivations in this appendix, let $K \geq 2$ be the number of domains and g_d the gradient contributed by domain d's sample losses with respect to the current W. It includes the shared domain weights and normalization, before the additional SINGLE-COMPONENT coefficient. Since $W = B + \sum_j P_j$, a sample has the same gradient with respect to W and each selected P_j. The active SINGLE-COMPONENT component at rollout step t is $a_t = 1 + ((t-1) \bmod K)$; its own domain's coefficient is 1 and each other domain's is $1/(K-1)$.

Gradient accumulation under SGD. Assume plain SGD with common learning rate $\eta > 0$, no clipping or decay, and the same normalized domain gradients evaluated at the same composite on the same batch. Write

W^+ for the composite after one update. Accumulating gradients into components as specified in Section 3.3 and substituting their SGD changes into Eq. (1) gives

$$W^+ - W = \begin{cases} -\eta \sum_d g_d, & \text{OWN}, \\ -\eta(K-1) \sum_d g_d, & \text{OTHER}, \\ -\eta \left(g_{a_t} + \dfrac{1}{K-1} \sum_{d \neq a_t} g_d \right), & \text{SINGLE-COMPONENT}. \end{cases} \tag{3}$$

In OWN, each domain gradient appears once in the sum of component updates; in OTHER, it appears $K-1$ times. ALTERNATE follows the rule of its current phase. These equalities hold at a fixed composite and batch; Adam and independent clipping generally change the multiplier relation.

Independent Adam histories. Assume zero weight decay. Initialize a shared student to $B + \sum_j P_{j,0}$ and give each domain an Adam state identical to its component's state in OWN. Match the data, teacher targets, student loss, normalization, clipping, learning rates, and per-domain update counters. The two implementations then have equal domain gradients and Adam increments. Adding those increments to the shared student gives the same parameters as recomposing the updated components. Induction extends equality to subsequent updates when they receive the same data and rollout random streams. This equivalence is in real arithmetic; finite-precision materialization and reduction order can introduce numerical differences.

If all components are active with common AdamW weight-decay coefficient λ, the combined decay term is $-\eta\lambda \sum_j P_j = -\eta\lambda(W - B)$. An equivalent shared-student control therefore decays toward B. Such a control can match OWN using per-domain moments without storing separate component tensors. In SINGLE-COMPONENT, only the active component is decayed, so reproducing this term requires that component's current parameters. For other routing rules, the corresponding optimizer-only control needs a state per destination component, which need not correspond to a single source domain.

Clipping before adaptive moments. For a gradient vector g, threshold $\tau > 0$, and numerical stabilizer $\epsilon_c = 10^{-6}$, define $C_\tau(g) = g \min(1, \tau/(\|g\|_2 + \epsilon_c))$. Independent clipping supplies $C_\tau(g_d)$ to each component's moments in OWN, whereas clipping the mixed gradient supplies $C_\tau(\sum_d g_d)$ to a shared optimizer. These operations generally differ even before Adam's nonlinear normalization. The parameter-update magnitude also depends on moment history and weight decay.

Gradient coordinates and component overlap. For this analysis, let h_d concatenate domain d's contributions to the K components before clipping. Under OWN,

$$h_d = (0, \ldots, g_d, \ldots, 0),$$

where g_d occupies block d and every other block is zero. Contributions from distinct domains have zero inner product because their nonzero entries occupy different blocks. In contrast, OTHER puts it in every block except d. Therefore, for distinct domains $d \neq e$,

$$\langle h_d, h_e \rangle = \begin{cases} 0, & \text{OWN}, \\ (K-2)\langle g_d, g_e \rangle, & \text{OTHER}. \end{cases} \tag{4}$$

The first identity follows from disjoint blocks; the second follows from the $K-2$ shared blocks. In SINGLE-COMPONENT, both domains contribute to the same active block. Their inner product equals $\langle g_d, g_e \rangle$ times their two additional sample coefficients defined above: $1/(K-1)$ if one domain is active, and $1/(K-1)^2$ otherwise. Summing component changes maps these separate blocks back to the same parameter coordinates in W.

Cross-domain effects through the composite. Let f_x be a scalar prediction statistic for a prompt x from any domain, twice continuously differentiable near W. If one component changes by δW, then

$$f_x(W + \delta W) - f_x(W) = \langle \nabla_W f_x(W), \delta W \rangle + O(\|\delta W\|_2^2).$$

The effect depends on the alignment between the update and the prediction gradient, since every domain uses the same W. For an explicit example, take a scalar composite parameter w and losses $\mathcal{J}_1(w) = (w-1)^2/2$ and $\mathcal{J}_2(w) = (w+1)^2/2$. At $w = 0$, their gradients are -1 and $+1$. Under OWN, their component vectors $(-1, 0)$ and $(0, +1)$ are orthogonal. Updating only the first component with SGD gives $w^+ = \eta$, increasing the second loss by $\eta + \eta^2/2 > 0$. Updating both with equal weights instead leaves the composite unchanged while the components move by $+\eta$ and $-\eta$. The example exhibits both cross-domain loss increases and cancellation after composition, despite disjoint gradient coordinates.

C Training Configuration

Table 5 specifies our Open-MOPD instantiation of the four routing modes with $K = 3$. The base and initialization experts are the SmolLM3-3B MixSFT, RL-Math, RL-Code, and RL-IF checkpoints from Open-MOPD. The data are its mixed Math, Code, and IF training set. Each component starts from the corresponding original expert minus MixSFT. Math and Code supervision both use a frozen TA $[1, 1, 1]$ merge of all three expert increments, while IF supervision uses the frozen original RL-IF. The mixture is selected for its stronger Math and Code scores in the available evaluations and because it can be built directly from public checkpoints without additional teacher training (Section 4.1). Thus initialization uses three distinct expert increments, independently of the two distinct supervising checkpoints.

Implemented distillation objective. The loss uses Open-MOPD's student-top-16 approximation and clipped policy surrogate (3, 14). Let n index samples, d_n their domains, and $h_{n\ell} = (x_n, y_{n,<\ell})$ their sampled prefixes. Let $S_{n\ell}$ contain the pre-update student's 16 highest-probability tokens at that prefix. Candidate identities and teacher scores remain fixed for the rollout batch. For $v \in S_{n\ell}$, define

$$\begin{aligned} s_{n\ell v} &= \log \pi_W(v \mid h_{n\ell}), \qquad t_{n\ell v} = \log \pi_{T_{d_n}}(v \mid h_{n\ell}), \\ \bar{p}_{n\ell v} &= \frac{\exp s_{n\ell v}}{\sum_{u \in S_{n\ell}} \exp s_{n\ell u}}, \quad A_{n\ell v} = \mathrm{sg}[\bar{p}_{n\ell v}(t_{n\ell v} - s_{n\ell v})] \,. \end{aligned}$$

Here sg stops gradients. Student-dependent terms are refreshed before each inner update when a rollout has multiple optimizer mini-batches; with one update, the precomputed advantages already use the same student. The advantage retains a separate value for each candidate. Only the student weights $\bar{p}$ are normalized over the candidate set; s and t are log probabilities under the full vocabulary distributions.

With detached reference log probability $s^{\mathrm{ref}}_{n\ell v}$, the candidate ratio and loss are

$$\begin{aligned} r_{n\ell v} &= \exp\big(\mathrm{clip}(s_{n\ell v} - s^{\mathrm{ref}}_{n\ell v}, -20, 20)\big) \,, \\ C(r, A) &= \max\{-rA, -\mathrm{clip}(r, 0.8, 1.28)A\}, \\ \phi(r, A) &= \begin{cases} \min\{C(r, A), -3A\}, & A < 0, \\ C(r, A), & A \geq 0. \end{cases} \end{aligned}$$

For the evaluated configuration (one mini-batch and one epoch per rollout), $s^{\mathrm{ref}} = \mathrm{sg}(s)$, so $r = 1$ numerically while its numerator remains differentiable. Thus ratio clipping does not activate in this configuration. With multiple mini-batches, the reference instead uses saved pre-update student log probabilities on the same candidate support.

Let $M_{n\ell}$ be the effective response mask, including any format mask, q_n the mode-specific sample coefficient from Section 3.3, and w_{d_n} the domain weight. Before domain packing, the loss for original

micro-batch m is

$$\mathcal{L}_m = \frac{1}{N_m} \sum_{n \in m} \sum_{\ell} M_{n\ell} \sum_{v \in S_{n\ell}} \phi(r_{n\ell v}, w_{d_n} q_n A_{n\ell v}), \qquad N_m = \sum_{n \in m, \ell} M_{n\ell} + \epsilon_m,$$

where ϵ_m is the mask-mean numerical stabilizer. Candidate losses are summed, then averaged over effective response tokens within each original micro-batch. Appendix A gives the packing correction and gradient accumulation semantics. No entropy bonus or additional reference-policy KL penalty is used.

Domain weights. All modes retain token-share balancing and gap-following allocation. On each global rollout batch, let f_d be domain d's observed response-token fraction and $b_d = (1/K)/f_d$. Let G_d be the mean of $|\sum_{v \in S_{n\ell}} A^{(0)}_{n\ell v}|$ over that domain's response tokens, where $A^{(0)}$ is the unweighted pre-update advantage. For the evaluated setting,

$$\bar{G} = \frac{\sum_d b_d G_d}{\sum_d b_d}, \qquad \widetilde{w}_d = b_d \,\mathrm{clip}\left(\frac{\max(G_d, 10^{-12})}{\max(\bar{G}, 10^{-12})}, 0.05, 20\right), \qquad w_d = \frac{\widetilde{w}_d}{\sum_e f_e \widetilde{w}_e}.$$

This uses exponent $\alpha = 1$, multiplies by the gap ratio without an initial-step anchor, and normalizes the token-weighted mean weight to one. Weights are computed once per rollout and held fixed during inner updates.

Sampling and updates. The top-16 restriction applies to loss computation. Training response generation uses stochastic sampling with temperature 1.0, top-p 1.0, and no top-k cutoff; periodic validation uses greedy argmax decoding with one response per prompt.

The sampler shuffles prompts within each domain pool, draws them cyclically, and shuffles the combined batch. Domain balance applies to the global batch; individual GPU shards need not contain equal domain counts.

The default rollout batch and optimizer mini-batch have equal size, yielding one optimizer update per rollout. Smaller optimizer mini-batches permit multiple updates on a rollout batch. Student reward refresh remains enabled in both cases; recomputation is needed for multiple inner updates, while the single-update case reuses advantages from the same student. Teacher targets and sampled trajectories are reused throughout. The 1:1:1 prompt quota determines sample counts, while token-share balancing and gap-following allocation determine domain loss weights. The default batch size is divisible by three, giving exactly equal counts in each batch. For a custom size not divisible by three, OWN, OTHER, and ALTERNATE rotate the integer remainder among domains across batches; SINGLE-COMPONENT requires exact equality and rejects such a size. The paper's SINGLE-COMPONENT mode corresponds to `single_delta` in the implementation.

AdamW uses first- and second-moment decay rates β_1 and β_2, respectively, and numerical stabilizer ϵ_{Adam}.

Model, data, and code provenance. Table 6 records the downloaded upstream revisions. Model and dataset repositories use the Hugging Face namespace `BytedTsinghua-SIA`. Training starts from `rl_prompt_mix/train.parquet` in the listed data revision. Exact complete-chat deduplication removes 721 rows; excluding groups with conflicting domain, source, answer, or IF verification rules removes another 5,294 rows. Splitting the remaining data with seed 42 holds out 512 prompts per domain, leaving 79,380 training and 1,536 validation prompts. An exact-prompt audit finds no overlap with the six evaluation inputs; this audit does not test paraphrases or altered chat wrappers. Evaluation uses that same data revision's `eval/math/aime24.parquet`, `eval/math/aime25.parquet`, `eval/code/livecodebench_v5.parquet`, `eval/code/livecodebench_v6.parquet`, `eval/if/ifeval_aligned.parquet`, and `eval/if/ifbench_test_aligned.parquet`. The upstream training framework is Open-MOPD; the evaluated run records local implementation commit `9bd7e97e7e64268de71e89cbd3d06ddb337d8519`. The manuscript source bundle does not include the training implementation or model checkpoints.

Table 5: Training configuration for domain-routed parameter updates. N_{GPU} is the number of GPUs. All four modes use the same initialization and sampling settings.

Setting	Value
Domains / component order	Math, Code, IF
Base / initialization	Frozen MixSFT; original expert-minus-base components, TA $[1, 1, 1]$ start
Rollout budget	200 steps
Global rollout / mini-batch	$24N_{\mathrm{GPU}}$ / $24N_{\mathrm{GPU}}$ prompts
Responses per prompt	1
Question sampling	1:1:1 global quota
Actor / student / teacher micro-batch	1 / 1 / 1 per GPU
Prompt / response maximum	2,048 / 16,384 tokens
Generation	Temperature 1.0; nucleus probability threshold 1.0; no top-k cutoff
Distillation loss support	Student-only top-16 candidates
Policy ratio clip / negative-advantage cap	$[0.8, 1.28]$ / 3
Policy epochs per rollout	1
Learning rate / scheduler	1.5×10^{-6} / constant
Adam $\beta_1, \beta_2, \epsilon_{\mathrm{Adam}}$	0.9, 0.999, 10^{-8}
Component weight decay / clip norm	0.01 / 1.0
Token-share balancing	Equal target token shares
Gap-following allocation	Mean absolute reward per domain; exponent $\alpha = 1$
Student reward refresh	Enabled for every optimizer mini-batch
Student training	SDPA, gradient checkpointing, static actor batching
vLLM rollout	Chunked prefill; eager disabled; memory fraction 0.5
vLLM sequence concurrency	64 per engine
Tensor / sequence parallel size	1 / 1
Actor / teacher parameter offload	False / True
Component and Adam state / gradient buffers	Host FP32 local shards / CUDA local shards
Seed	42

Table 6: Upstream artifact revisions used for initialization and data. Repository names in the first five rows are relative to `BytedTsinghua-SIA` on Hugging Face. The last row is the GitHub repository.

Repository	Revision
`Open-MOPD-SmolLM3-3B-MixSFT`	`c9e7bad031667828656ead188d7e8ea162c048a4`
`Open-MOPD-SmolLM3-3B-RL-Math`	`5e901bb626b69d711074d2832c41ce1aa4232da8`
`Open-MOPD-SmolLM3-3B-RL-Code`	`e2ce9beec52381350edcdaefebfd08ea67c21b42`
`Open-MOPD-SmolLM3-3B-RL-IF`	`8948051a805883e2db988cae4522de3c29d1d112`
`Open-MOPD-Data`	`9e897efe3257599d4300e2d5ee865a1cc714af87`
`BytedTsinghua-SIA/Open-MOPD`	`4809a96cf85a869106ff0ff3f37d0a51e12010ae`

D Evaluation Metrics

The TA, orthogonalized-merge, and OWN step-20 evaluations use AIME 2024 and AIME 2025 (30 questions each), LiveCodeBench v5 and v6 (7) (167 and 175 questions), IFEval (20) (541 prompts), and IFBench test (12) (300 prompts). Each AIME question has 64 sampled responses, each code question has 10, and each instruction-following prompt has one. The average correctness over these responses estimates single-response performance. Training uses one response per prompt.

All three local models use the same benchmark inputs, decoding protocol, and scoring entrypoint. TA and OWN generation used 16 GPUs, while the orthogonalized merge used 32. Generation does not set a per-request sampling seed. Each result is from one evaluation run, so small score differences include sampling and batching variation. Evaluation uses temperature 0.6 for Math and 1.0 for Code and IF, top-p 0.95, no top-k cutoff, a 32,768-token context limit, and a 30,000-token output limit. Scoring fixes Python, NumPy, and language-detection seeds to zero. The OWN results use the exported BF16 student from the same step-20 checkpoint analyzed in Appendix E.

For benchmark b with N_b prompts and n_b sampled responses per prompt, let $i = 1, \ldots, N_b$ index prompts and $q = 1, \ldots, n_b$ index sampled responses. The indicator $z_{biq}(W)$ equals one if response q to prompt i, generated by model W, satisfies benchmark b's correctness criterion, and zero otherwise. The estimates $\widehat{s}_b$, $\widehat{s}_d$, $\widehat{s}_{\mathrm{macro}}$, and $\widehat{s}_{\mathrm{worst}}$ denote benchmark, domain, macro-average, and minimum domain scores, respectively:

$$\widehat{s}_b(W) = \frac{100}{N_b} \sum_{i=1}^{N_b} \frac{1}{n_b} \sum_{q=1}^{n_b} z_{biq}(W),$$

$$\widehat{s}_d(W) = \frac{1}{2} \sum_{b \in \mathcal{B}_d} \widehat{s}_b(W), \qquad \widehat{s}_{\mathrm{macro}}(W) = \frac{1}{3} \sum_{d} \widehat{s}_d(W), \qquad \widehat{s}_{\mathrm{worst}}(W) = \min_d \widehat{s}_d(W). \tag{5}$$

Here $\mathcal{B}_d$ contains the two benchmarks for domain d; all scores are percentages. This weighting gives each domain equal importance. IFEval and IFBench use strict prompt-level correctness.

E Additional Weight-Geometry Results

Measurement scope. We measure all 327 serialized tensors per model, covering 3,337,766,912 stored coordinates, with FP64 differences and accumulation. This convention counts both tied embedding and output-head tensors.

The original expert increments are nearly orthogonal. We collect the initial expert increments defined in Section 3.1 as columns of $D = [\mathrm{vec}(P_{1,0}), \mathrm{vec}(P_{2,0}), \mathrm{vec}(P_{3,0})]$, in Math, Code, and IF order. The operator vec concatenates the stored parameter tensors in a fixed order, so $D \in \mathbb{R}^{n_{\mathrm{par}} \times 3}$, where n_{par} is the number of stored coordinates counted above. For nonzero vectors z, z', we use $\cos(z, z') = \langle z, z' \rangle / (\|z\|_2 \|z'\|_2)$ for cosine similarity. The full-weight cosines are 0.02100095 for Math–Code, 0.00020855 for Math–IF, and 0.00035706 for Code–IF, showing near orthogonality of the original expert increments in global parameter space.

Projection onto the expert span. For a checkpoint W, decompose its flattened increment u as

$$u = \mathrm{vec}(W - B), \qquad a = (D^\top D)^\dagger D^\top u, \qquad r = u - Da, \qquad \rho_\perp = \frac{\|r\|_2^2}{\|u\|_2^2}. \tag{6}$$

Here $\dagger$ denotes the Moore–Penrose pseudoinverse. The vector $u \in \mathbb{R}^{n_{\mathrm{par}}}$ is the checkpoint's change from the base; $a = (a_{\mathrm{Math}}, a_{\mathrm{Code}}, a_{\mathrm{IF}})^\top \in \mathbb{R}^3$ contains its least-squares projection coefficients, and $r \in \mathbb{R}^{n_{\mathrm{par}}}$ is the residual orthogonal to the columns of D. The fraction $\rho_\perp$ measures squared norm outside the original expert span and is undefined when $u = 0$, as for the unchanged base.

Angle references. For nonzero vectors v and r, the reported angle is $\arccos(\cos(v, r))$, converted to degrees. In Table 4, composite-model rows use $u_{\mathrm{TA}} = \mathrm{vec}(W_{\mathrm{TA}} - B)$ as the reference. Component-retention rows compare $P_{j,s}$ with $P_{j,0}$; pairwise rows compare $P_{j,s}$ with $P_{k,s}$ at the same step. Tensors are flattened using the same coordinate order. The saved TA–Final and TA–orthogonal cosines give angles of $84.3962°$ and $1.6618°$, respectively; the Own BF16 student's stored cosine to its TA initialization gives $12.8600°$. MixSFT's zero increment has no defined direction.

Component rotation and energy growth. Write $z_{j,s}$ for the flattened saved component for domain j at reported step s, so $z_{j,0} = \mathrm{vec}(P_{j,0})$. We measure its rotation from initialization and energy as

$$\theta_{j,s} = \arccos\big(\cos(z_{j,s}, z_{j,0})\big), \qquad E_{j,s} = \|z_{j,s}\|_2^2, \qquad \gamma_{j,s} = 100\left(\frac{E_{j,s}}{E_{j,0}} - 1\right).$$

Angles are reported in degrees, and $\gamma_{j,s}$ is the percentage growth of the component's squared norm from initialization.

Composite increment measurements. For a saved run, let $u_s^{\mathrm{comp}} = \sum_j z_{j,s}$ be the mathematical sum of the saved FP32 components, accumulated in FP64. Its total energy relative to the frozen base is

$$E_s^{\mathrm{sum}} = \|u_s^{\mathrm{comp}}\|_2^2 = \sum_j E_{j,s} + 2\sum_{j<k}\langle z_{j,s}, z_{k,s}\rangle.$$

The cross terms capture pairwise interactions between the components. For BF16 student parameters W_s^{BF16}, measure $u_s^{\mathrm{BF16}} = \mathrm{vec}(W_s^{\mathrm{BF16}} - B)$. Apply Eq. (6) to each representation using the same original expert matrix D.

Model identities. The historical comparison covers 24 models: the common MixSFT base, original RL-Math/RL-Code/RL-IF specialists, TA merge, symmetric orthogonalized merge, public Open-MOPD Final, and 17 local training checkpoints. The public Final is step 200 at revision `228a146a5d95f001360573 47ac4810e6635061b6`. We additionally analyze the saved components and BF16 student of one step-20 OWN checkpoint, giving 25 model states in total.

Counting and precision. The main table follows the original serialized-coordinate convention, including both tied embedding and output-head tensors. Counting that matrix once gives 3,075,098,624 coordinates and Final outside energy of 97.0078%; restricting the calculation to transformer layers gives 96.9944%. Thus the main 97.02% observation is insensitive to this counting choice. The historical statistics compare stored BF16 checkpoints. The new OWN analysis additionally reads the native FP32 components; all subtraction and accumulation use FP64.

Alignment of the Final checkpoint. TA and the orthogonalized merge remain almost entirely in the original expert span; their small residuals reflect finite-precision export. In contrast, 97.02% of Final's increment energy lies outside that span, and its cosine with the TA increment is only 0.09765. Its cosines with the individual Math, Code, and IF increments are 0.01696, 0.03034, and 0.16914, respectively. Reconstructing the released Final checkpoint therefore requires parameter directions beyond the three original expert increments.

Distribution of parameter changes. Let $W^{[m]}$ and $B^{[m]}$ denote the checkpoint and base tensors in layer m, with layers indexed from zero. Ranking layers by $\|W^{[m]} - B^{[m]}\|_2 / \|B^{[m]}\|_2$ places L16, L15, L14, L13, and L12 highest for TA, but L35, L34, L19, L33, and L18 for Final. Final's relative changes thus emphasize later layers. MLP down/up/gate projections account for 32.75/28.22/20.70% of Final's increment energy, compared with 30.07/29.32/25.72% for TA; attention output projections account for 10.53% versus 7.49%. These energy shares also depend on module size. The largest individual Final–TA difference, L35 down projection, accounts for only 1.44% of their total squared difference: the change is distributed across many tensors.

Construction of the orthogonalized merge. Let $N_D = \mathrm{diag}(\|P_{1,0}\|_2, \|P_{2,0}\|_2, \|P_{3,0}\|_2)$ be the diagonal matrix of original expert-increment norms, ordered Math, Code, IF. The matrix $C = N_D^{-1} D^\top D N_D^{-1}$ contains their pairwise cosines. For these three linearly independent directions, C is positive definite; $C^{-1/2}$ denotes its symmetric inverse square root. The matrix of orthogonalized directions is

$$D_{\mathrm{orth}} = D N_D^{-1} C^{-1/2} N_D, \qquad D_{\mathrm{orth}}^\top D_{\mathrm{orth}} = N_D^2.$$

Let unvec invert the parameter flattening and $\mathrm{BF16}$ denote elementwise rounding to bfloat16. The comparison checkpoint is $W_{\mathrm{orth}} = \mathrm{BF16}(B + \mathrm{unvec}(D_{\mathrm{orth}}\mathbf{1}))$, where $\mathbf{1} = (1,1,1)^\top$. Before rounding, its effective coefficients on the original experts are approximately $(0.984510, 0.992992, 0.999525)$. The coefficients measured from the saved checkpoint in Table 4 differ because of BF16 rounding. Its increment cosine with TA is 0.9995794; 10,604,757 stored coordinates change between the two exports.

Additional updates after TA initialization. For the TA-initialized reward-v4 run, let W_s denote the saved checkpoint at reported training step s, and W_{TA} the saved initial TA merge. Define its initial increment as $u_{\mathrm{TA}} = \mathrm{vec}(W_{\mathrm{TA}} - B)$. Distinguish the total increment $u_s = \mathrm{vec}(W_s - B)$ from the additional training update $v_s = \mathrm{vec}(W_s - W_{\mathrm{TA}})$. Apply Eq. (6) to each vector with the same original expert-direction matrix D. Table 7 records both denominators; the large residual share of a small additional update can coexist with a small residual share of the total increment.

Table 7: TA-initialized MOPD checkpoints. Total outside energy uses $\|u_s\|_2^2$; update outside energy uses $\|v_s\|_2^2$. Coefficients describe v_s, ordered Math, Code, IF. These checkpoints share one reward-v4 training trajectory.

Step	$\cos(u_s, u_{\mathrm{TA}})$	Total outside (%)	Update coefficients	Update outside (%)
30	0.98591	2.86	$(-0.00673, -0.00812, -0.00759)$	99.79
80	0.96782	6.39	$(-0.01505, -0.01902, -0.00483)$	99.58
100	0.96458	7.02	$(-0.01624, -0.01987, -0.00460)$	99.58

Negative coefficients denote a small reverse projection along the specified expert directions. The other 14 local checkpoints start from MixSFT and come from the abc, acos_1101, no_neg, and reward-v4 runs, with different objectives and budgets. Their total increments have 99.28–99.62% of their energy outside the original expert span.

Source and checks for the step-20 OWN components. The run is `own-task_sum-nodes4-gpus8-globalbsz768-20261106-223909-393560`, with native components from `checkpoints/global_step_20/actor` and the composite from `eval/global_step_20/student_hf`. We read all 32 optimizer shards, verifying step 20 and 20 updates for each component on every rank. The global batch and mini-batch are both 768, so each rollout has one optimizer update. The seed is 42, with the remaining training settings in Table 5. Initialization uses the prepared FP32 differences between the three original experts and MixSFT.

We verify every stored base coordinate against MixSFT and reconstruct the student by adding the three FP32 components to the base in the training implementation's order. Casting that result to BF16 matches every coordinate of the exported student. The initial BF16 composite is reconstructed by the same procedure using the original components; its 0.057632% outside energy reproduces the independent TA measurement. The initial pairwise cosines also reproduce the historical measurements. The source paths, input file sizes and modification times, full Gram statistics, counting controls, and analysis script hash are recorded in `weight_analysis.json`.

The largest absolute cosine between components changes from 0.021001 at initialization to 0.042120 at step 20.

The current FP32 component sum has projection coefficients $(0.998552, 0.998477, 1.000223)$, close to the initial $(1, 1, 1)$, while 10.52% of its total increment energy is outside their span. The BF16 export has coefficients $(0.999213, 0.999464, 1.000219)$ and 4.90% outside energy. The initial outside-energy fractions are 0.0000% for the FP32 component sum and 0.0576% for the BF16 composite, whose initial weights are reconstructed in the training materialization order. Its total increment energy grows by 5.13%, compared with 11.48% for the FP32 component sum. The FP32 sum rotates by 18.93° from its initialization, with energy increasing from 2.585090 to 2.881931 and a step-20 outside-energy fraction of 10.5217%.

Under the serialized-coordinate convention, the current Math, Code, and IF components have 10.4794%, 2.8788%, and 36.9234% of their energy outside the original expert span, respectively. For their additional updates $z_{j,20} - z_{j,0}$, the fractions are 99.9803%, 99.9552%, and 99.9525%. The summed additional update has 99.9984% outside energy for the FP32 components and 99.9999% for the BF16 composite difference. These percentages normalize by the respective *update* energies, rather than the *current total increment* energies used in Table 4.

The pairwise cosines of the accumulated component updates are 0.180884, -0.195586, and -0.094734 for Math–Code, Math–IF, and Code–IF. Counting the tied embedding/output matrix once changes the current FP32 composite's outside energy from 10.5217% to 10.3662%; restricting to transformer layers gives 10.2093%. The corresponding update outside fractions remain approximately 99.9983%, and the maximum absolute current-component cosine remains below 0.043 under both controls.

Runtime configuration. In the inspected Transformers runtime, the RoPE base resolves to 2×10^6 for MixSFT/Final and 5×10^6 for TA/orthogonal. Static weight statistics are independent of this setting, but behavioral comparisons also depend on runtime configuration, prompts, decoding, and the evaluation protocol.